AI EXPLAINED · EPISODE 2

How an LLM actually thinks

We build a tiny language model, train it, open its weights and watch it predict.

From zero to building an AI agent

Every answer ChatGPT ever gave you was written one piece at a time.

`guess → add → guess → add → …`

LAST EPISODE

Two ideas to carry in

Train → Learn → Predict

Teachable Machine learned left vs right from examples, not rules.

An LLM predicts what comes next

Same idea, trained on huge amounts of text.

Today: **how** does it predict?

LAB · TINY LLM LAB

A real language model, small enough to see

“My puppy name is River. River likes Ruby.”

1 Train it on this text

2 Save its weights

3 Freeze and predict

kagitha-dev.github.io/tiny-llm-lab

LAB · STEPS 1 AND 2 · TRAIN AND SAVE

Watch the numbers change

Before: the grid is random numbers. The model knows nothing.

Train: each round, the real next word's number goes up. "puppy" after "my" turns blue.

Error: the loss number falls round after round.

Save: the whole grid is written to a file.

Tip: open "What's happening" to show one real nudge, with numbers

LAB · STEP 3 · FREEZE AND PREDICT

Chances, then a draw

Type “my”: it reads the saved “my” row, and “puppy” wins.

Type “river”: “likes” and “.” are both possible, about half each.

Add a token by chance: click a few times. The answer changes.

Generate: predict, add, repeat. A sentence writes itself.

Tip: open “Why doesn’t it always pick the top token?” and click Draw 100 times

LAB · TEXT TO NUMBERS

How your words get in



Play all: watch “My puppy name is River.” become numbers.

Type “godfather”: letters here, but god + father in GPT-4o.

LAB · HOW A WEIGHT IS MADE

Follow one number: “my → puppy”

START (RANDOM)	ROUND 1	ROUND 2	ROUND 3	ROUND 13
−0.44	0.02	0.45	0.85	2.89

Each round: check the chance of “puppy”, see how much is missing to 100%, **add half of it**.

WHAT IS WHAT

Now let’s name what you just saw.

Model · Weights · Tokens · Embeddings · Prediction · Temperature · Context window · Inference

Model

A system whose numbers are **learned from data**, so it can recognise patterns, make predictions or generate content.



Weights (parameters)

The **learned numbers** inside the model. Training adjusts them; the result is saved in a file.

TINY LLM LAB	GPT-2 SMALL (2019)	LLAMA 3 8B (2024)
64	**124 million**	**8 billion**

Saved as files like **.safetensors** or **.gguf**. Nobody types a weight in.

Tokens and the tokenizer

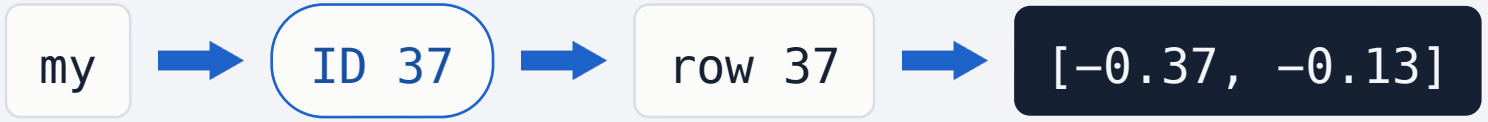
A **token** is a piece of text the model works with. The **tokenizer** cuts text into tokens and gives each an ID.

My 5444	·puppy 50715	·name 1308	·is 382	·River 13728	. 13

GPT-4o's real tokenizer, above. Common words are one token; rare ones split: "godfather" → **god + father**. About 200,000 tokens in its vocabulary.

Embedding

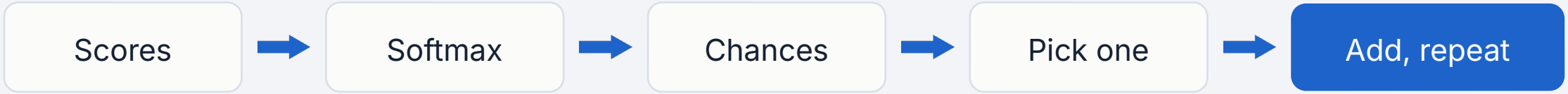
The list of learned numbers a token's ID picks from the **embedding table**.
Like a phone book: the ID is the row number.



Lab: 2 numbers per token. GPT-2: 768. Llama 3 8B: 4,096. After training, words used in similar places get similar numbers.

Next-token prediction

The model gives **every token a score**, softmax turns the scores into chances that add up to 100%, one token is picked, added, and it repeats.



It never decides “the answer”. It only says how likely each next token is.

Sampling and temperature

Sampling picks by chance, like drawing a marble: 57 “cat”, 43 “puppy”. **Temperature** changes the jar.

Temperature	The jar	Feels
Low (0.2)	cat ≈ 80%, puppy ≈ 20%	Predictable
Normal (1.0)	cat 57%, puppy 43%	Balanced
High (2.0)	Closer to even; rare tokens get a chance too	Varied

Top-k limits how many tokens can be drawn at all. Top-k 1 = always the top token.

Context window

How many tokens the model can **see at once**: your prompt plus everything written so far. Anything outside it, the model can't see.

TINY LLM LAB	GPT-2 (2019)	MANY MODELS TODAY
1 token	**1,024**	**128,000+**

Our lab sees only the last word, so after "river" it can't know who you're talking about.

Training vs inference

Training

Weights **change**, nudge by nudge

Done once, before release

Weeks, thousands of chips

Inference

Weights are **frozen**, only read

Every time you send a prompt

Seconds

Chatting doesn’t teach the model. Only training changes its weights.

Pre-training

The first, huge training stage: the model learns to **predict the next token** on enormous amounts of text.

THE RESULT: A BASE MODEL

Great at continuing text. Not yet good at following instructions or chatting.

IN OUR LAB

Clicking Train on random weights is pre-training in miniature.

Post-training and fine-tuning

Post-training teaches the base model to follow instructions and be helpful.
Fine-tuning is training a trained model further on more specific data.

Supervised fine-tuning

Learn from example conversations

Preference training

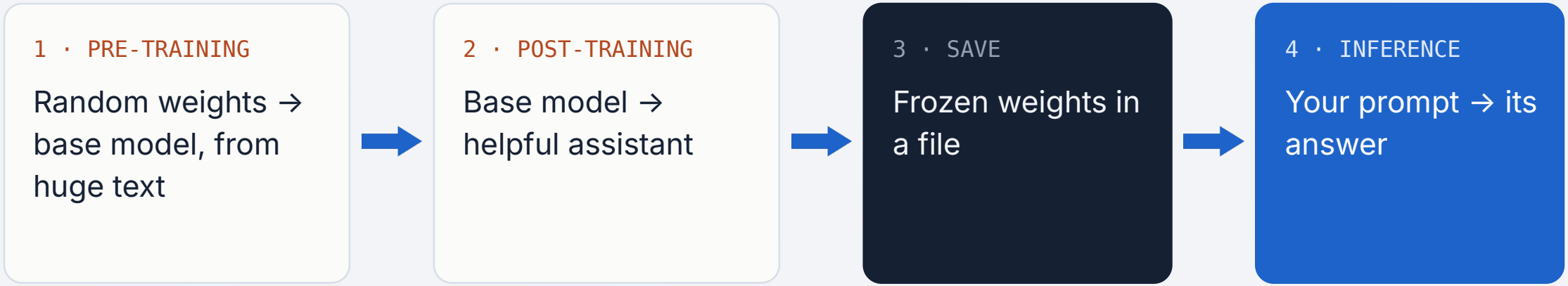
Learn which answers people prefer

Reinforcement learning

Get rewarded for good results

In our lab: **Add** a sentence, then **Train** again on top of the trained weights.

The life of an LLM



Steps 1 and 2 change the weights. Step 4, every chat, only reads them.

WHY IT CAN BE WRONG

Hallucination

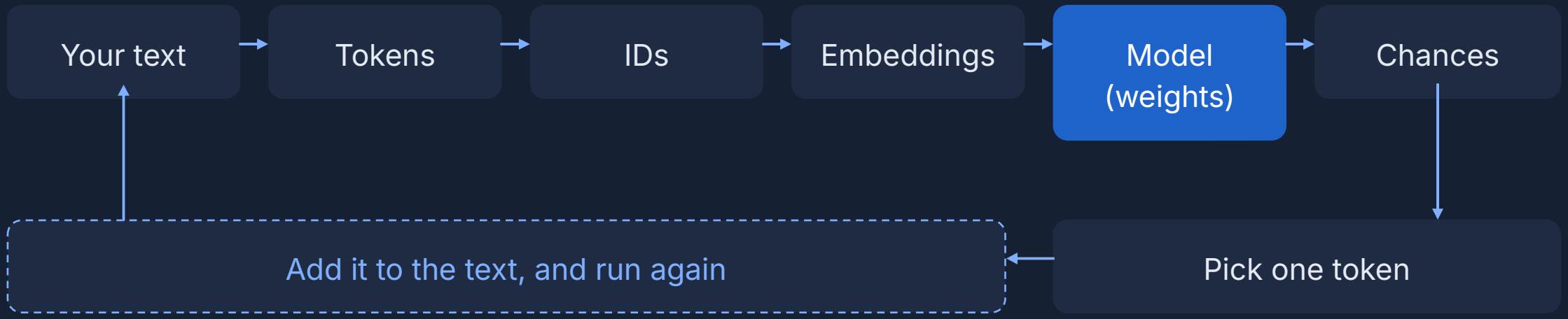
A **confident answer that isn't true**. It predicts what's *likely*, not what's *true*, and it always picks something.

TRY IT IN THE LAB · TRAINING TEXT

```
My puppy name is River. My cat name is Ruby. River likes Ruby.
```

"my puppy name is" → river 50%, ruby 50% → it can write **"my puppy name is ruby."** Every step looked fine.

How an LLM thinks, in one picture



Tokenizer, embeddings, weights, softmax, sampling: **you have now seen every one.**

NEXT · EPISODE 3

[Episode 3 title]

Try the labs yourself: kagitha-dev.github.io/tiny-llm-lab

Comment below: which lab made it click for you?

Like · Subscribe · Turn on notifications